

Quizfy Features and Architecture Workflow

This document explains what Quizfy does, where the main features live, and how the application flows from user action to backend processing.

1. Product Overview

Quizfy is a Django-based educational quiz platform for teachers and students. Teachers create quizzes, manage questions, share quizzes through links or QR codes, review submissions, export reports, and run class performance analytics. Students log in, take assigned quizzes, submit answers, and view their results.

The app is designed around real classroom workflows:

- Teacher creates an assessment.
- Student accesses the assessment securely.
- Student submits answers.
- Teacher reviews results.
- Teacher exports reports.
- Teacher can upload the assessment PDF and class Excel results to find weak class and student topics.

2. Main Features

Teacher Features

- Teacher signup, login, logout, and password management.
- Create quizzes.
- Edit quiz settings.
- Add, edit, and delete questions.
- Support multiple question types:
 - Multiple choice
 - True/false
- File upload/manual review questions
 - Organize quizzes into subject folders.
 - Move quizzes between folders.
 - Enable or disable quizzes.
 - Generate QR codes for quizzes.
- View student submissions.
- Grade file-upload questions.
- Allow extra attempts for students.
- Export quiz submissions to Excel.
- Export folder-level reports.
- Export student-level reports.
- Export quiz questions as PDF.
- Run class performance analytics from PDF + Excel uploads.

Student Features

- Student signup, login, logout, and password management.
- Join quizzes by code or QR flow.
- Take active quizzes.

- Answer questions online.
- Submit quiz attempts.
- See quiz result feedback after submission.
- View previous submissions from the student dashboard.

Reporting Features

- Quiz submission Excel export.
- Folder boxes Excel export.
- Student folder report export.
- Per-question correctness columns such as Q1, Q2, Q3.
- Correct/wrong values represented as:
 - 1 = correct
 - 0 = wrong or unanswered
- PDF quiz export for teacher reference.

Class Performance Analytics / RAG Feature

The analytics feature lets a teacher upload:

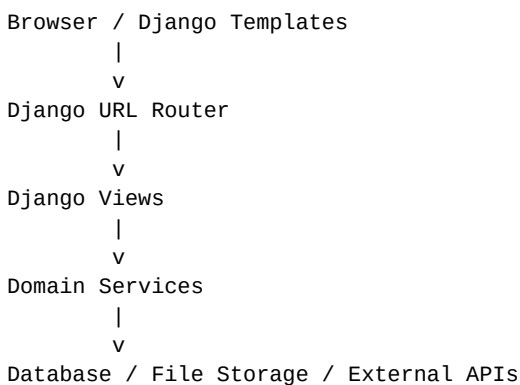
1. A PDF containing assessment questions.
2. An Excel/CSV/TSV file containing class results.

The system then returns:

- Class-level weak topics.
- Student-level weak topics.
- Evidence question numbers.
- Mistake counts.
- Attempt counts.
- Error rates.
- Weakness scores.
- Confidence values.
- Teacher-friendly explanations.

The analytics workflow is class-first. It analyzes the entire class by default, then also provides per-student summaries.

3. High-Level Architecture



In Quizfy, most user-facing pages are rendered with Django templates. The backend is responsible for authentication, quiz rules, grading, exports, file processing, analytics, and calls to external services.

4. Core Project Structure

```
Quizfy/
+-- quizzes/
|   +-- models.py
|   +-- views.py
|   +-- forms.py
|   +-- urls.py
|   +-- signals.py
|   +-- templates/
|       +-- quizzes/
|
+-- quizz_app/
|   +-- settings.py
|   +-- urls.py
|   +-- wsgi.py
|   +-- safe_logging.py
|
+-- performance_analytics/
|   +-- config.py
|   +-- models.py
|   +-- prompts.py
|   +-- storage.py
|   +-- startup_check.py
|   +-- services/
|       |   +-- pdf_processor.py
|       |   +-- topic_classifier.py
|       |   +-- excel_processor.py
|       |   +-- analyzer.py
|       |   +-- vector_store.py
|   +-- tests/
|
+-- scripts/
|   +-- security_scan.py
|
+-- docs/
+-- manage.py
+-- requirements.txt
+-- render.yaml
+-- .env.example
+-- .gitignore
```

5. Main Files and Responsibilities

quizzes/models.py

Defines the main database models for quizzes, questions, submissions, answers, folders, student profiles, and attempt permissions.

This file represents the app's persistent academic data.

quizzes/views.py

Contains the main Django view functions.

It handles:

- Teacher dashboards.
- Student dashboards.
- Quiz creation.
- Question management.

- Quiz taking.
- Submission handling.
- Grading.
- Excel exports.
- PDF exports.
- The analytics upload page.

The analytics entry point is:

`performance_analytics_dashboard`

quizzes/forms.py

Defines Django forms used for login, signup, quiz creation, question creation, folder movement, settings, password changes, and file-upload submissions.

quizzes/urls.py

Maps app URLs to Django views.

Important analytics routes:

```
/teacher/analytics/
/teacher/performance-analytics/
/analytics/
/a/
```

quizzes/templates/

Contains the HTML pages rendered by Django.

Important analytics template:

`quizzes/templates/quizzes/performance_analytics.html`

This file renders the upload UI and displays class/student analytics results.

quizz_app/settings.py

Main Django settings file.

It controls:

- Installed apps.
- Middleware.
- Database settings.
- Static and media files.
- Allowed hosts.
- CSRF trusted origins.
- Logging.
- Deployment-related settings.

quizz_app/urls.py

Root URL configuration.

It includes the `quizzes.urls` routes and admin routes.

quizz_app/safe_logging.py

Provides redaction-safe logging helpers. It redacts common secret-looking values from logs as a safety layer.

This protects against accidental logging of sensitive values.

scripts/security_scan.py

Scans the repository for secret leakage risks.

It checks for risky patterns such as:

- Printing environment variables.
- OpenAI key-shaped strings.
- SendGrid key-shaped strings.
- Bearer token-looking strings.
- Header dumps.
- Cookie/token/auth logging.
- Secret-like JSON values.

6. Quiz Creation Workflow

```
Teacher logs in
      |
      v
Teacher opens quiz dashboard
      |
      v
Teacher creates quiz
      |
      v
Teacher adds questions
      |
      v
Teacher activates quiz
      |
      v
System generates quiz link / QR code
```

Important files:

```
quizzes/views.py
quizzes/forms.py
quizzes/models.py
quizzes/templates/quizzes/create_quiz.html
quizzes/templates/quizzes/create_question.html
```

7. Student Quiz-Taking Workflow

```
Student opens quiz link or scans QR code
      |
      v
If not logged in, student is redirected to login
      |
      v
Student opens quiz page
      |
      v
```

Student answers questions
|
v
Student submits quiz
|
v
Backend grades objective questions
|
v
Student sees result page

Important files:

quizzes/views.py
quizzes/templates/quizzes/take_quiz.html
quizzes/templates/quizzes/quiz_result.html
quizzes/models.py

8. Submission and Grading Workflow

Student submits quiz
|
v
Submission record is created
|
v
Answer records are created
|
v
Objective questions are graded automatically
|
v
File-upload questions wait for teacher grading
|
v
Teacher reviews submission
|
v
Teacher grades manual items if needed

Important files:

quizzes/views.py
quizzes/models.py
quizzes/templates/quizzes/quiz_submissions.html
quizzes/templates/quizzes/grade_submission.html

9. Excel Export Workflow

Teacher clicks Export Excel
|
v
Django loads quiz submissions
|
v
Backend creates workbook with openpyxl
|
v
Workbook includes student info, score, percentage, Q1/Q2/Q3 columns
|

Browser downloads the Excel file

The per-question columns are important because analytics can later read:

Q1 = 1 or 0
Q2 = 1 or 0
Q3 = 1 or 0

Important file:

quizzes/views.py

Important functions include the Excel export views.

10. PDF Export Workflow

```
graph TD
    Teacher[Teacher clicks Export PDF] --> Backend[Backend loads quiz questions]
    Backend --> Backend2[Backend generates a formatted PDF]
    Backend2 --> PDF[PDF includes numbered questions]
    PDF --> Browser[Browser downloads the PDF]
```

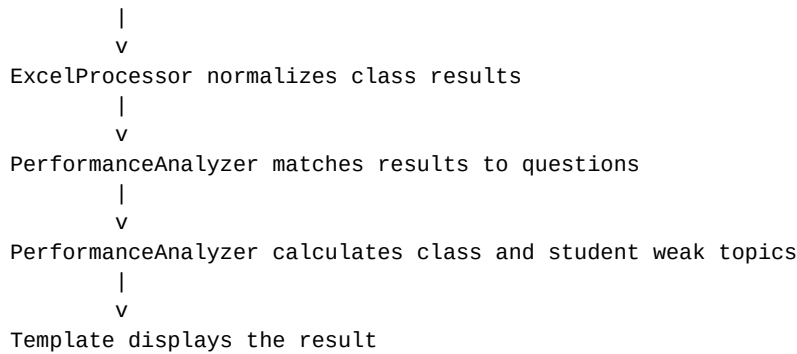
The numbered questions are important for analytics because the PDF question numbers must match Excel columns such as Q1, Q2, and Q3.

Important file:

quizzes/views.py

11. Class Performance Analytics Workflow

```
graph TD
    Teacher[Teacher opens /teacher/analytics/] --> Teacher2[Teacher uploads PDF assessment + Excel class results]
    Teacher2 --> Frontend[Frontend sends FormData POST to Django]
    Frontend --> Django[Django receives files]
    Django --> PDFProcessor[PDFProcessor extracts questions from PDF]
    PDFProcessor --> TopicClassifier[TopicClassifier assigns topic/subtopic to each question]
    TopicClassifier --> VectorStore[VectorStore stores question evidence for RAG]
```



12. Analytics File Responsibilities

performance_analytics/config.py

Reads analytics environment variables and validates required settings.

Important values:

```
OPENAI_API_KEY
OPENAI_MODEL
VECTOR_DB_TYPE
VECTOR_DB_PATH
UPLOAD_DIR
LLM_BATCH_SIZE
CONFIDENCE_THRESHOLD
```

The API key is read only from the backend environment.

performance_analytics/models.py

Defines typed data models for analytics.

Important models:

```
ExtractedQuestion
ClassifiedQuestion
StudentResult
ClassWeakTopic
StudentWeakTopic
ClassSummary
StudentSummary
AnalysisRequest
AnalysisResponse
```

performance_analytics/services/pdf_processor.py

Reads a PDF and extracts structured question records.

It supports question styles such as:

```
1. Question text
Q1. Question text
Question 1: Question text
QUESTION 1
```

performance_analytics/services/topic_classifier.py

Assigns each question a topic and subtopic.

It can use OpenAI through the backend. If OpenAI is temporarily unavailable, it has a deterministic fallback for common math topics.

performance_analytics/services/excel_processor.py

Reads Excel/CSV/TSV class results.

It supports:

```
student_id | student_name | Q1 | Q2 | Q3
```

and the app's exported Excel format:

```
Student Full Name | University ID | Score | Total | Percentage | Q1 | Q2 | Q3
```

It converts wide rows into normalized per-question records.

performance_analytics/services/analyzer.py

Calculates class and student weak topics.

It matches:

```
Excel Q1 -> PDF Question 1  
Excel Q2 -> PDF Question 2  
Excel Q3 -> PDF Question 3
```

Then it calculates:

- Mistakes.
- Attempts.
- Error rate.
- Students affected.
- Weakness score.
- Confidence.
- Evidence question numbers.

performance_analytics/services/vector_store.py

Stores classified questions for RAG-style evidence retrieval.

It stores metadata such as:

```
question_id  
question_number  
topic  
subtopic  
source_pdf  
page_number
```

This lets the system connect explanations back to real uploaded questions.

performance_analytics/prompts.py

Contains internal prompts used for topic classification and weak-topic explanations.

performance_analytics/storage.py

Stores uploaded files and JSON snapshots of questions, results, and analyses.

performance_analytics/startup_check.py

Validates analytics startup requirements.

performance_analytics/example_workflow.py

Developer-oriented example showing how the analytics pipeline can be run outside the UI.

13. Analytics Data Flow Example

PDF:

Q1: Solve $2x = 10$
Q2: Solve $x + 3 = 9$
Q3: Find the area of a rectangle

Topic classification:

Q1 = Algebra > Linear Equations
Q2 = Algebra > Linear Equations
Q3 = Geometry > Area

Excel:

student_id	student_name	Q1	Q2	Q3
S001	Student A	0	0	1
S002	Student B	1	0	1
S003	Student C	0	1	0

Normalized backend records:

S001 Q1 wrong
S001 Q2 wrong
S001 Q3 correct
S002 Q1 correct
S002 Q2 wrong
S002 Q3 correct
S003 Q1 wrong
S003 Q2 correct
S003 Q3 wrong

Combined result:

Algebra mistakes = Q1 + Q2 mistakes
Geometry mistakes = Q3 mistakes

UI result:

Top class weak topic:
Algebra > Linear Equations
Evidence: Q1, Q2

14. Security Architecture

Security rules in this app:

- API keys are never hardcoded.
- API keys are loaded from backend environment variables.
- Frontend never receives the OpenAI API key.
- .env is ignored by Git.
- .env.example contains placeholders only.
- Logs are redacted through safe logging.
- The security scanner checks for secret leakage risks.
- CSRF protection is enabled.
- Analytics upload uses Django CSRF protection.

Important files:

```
quizz_app/safe_logging.py
scripts/security_scan.py
.gitignore
.env.example
performance_analytics/config.py
```

15. Deployment Workflow

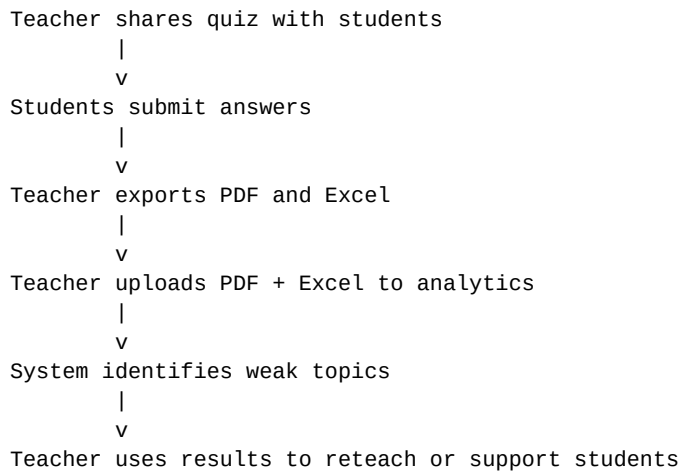
```
Code pushed to GitHub
|
v
Render deploy starts
|
v
Dependencies install from requirements.txt
|
v
Django app starts with gunicorn
|
v
Environment variables are read from Render settings
|
v
App serves teacher/student workflows
```

Important deployment files:

```
render.yaml
requirements.txt
build.sh
Dockerfile
quizz_app/settings.py
```

16. End-to-End User Workflow

```
Teacher creates quiz
|
v
Teacher adds questions
|
v
```



17. Why This Architecture Works

The app keeps responsibilities separated:

- Django views coordinate requests.
- Templates display pages.
- Models define database data.
- Forms validate user input.
- Analytics services do specialized processing.
- Security tools protect secrets.
- Docs explain setup and workflows.

This makes the project easier to maintain, debug, and extend.